UNK_DeadDrop: An Artifact-Based Study of Recruitment Malware, Credential Collection, and Operator Infrastructure

Ilja Krasnjanski

Independent researcher

ORCID: 0009-0007-1039-5930

25 October 2026 · Preprint

Abstract

Recruitment-themed malware connects developer targeting with delivery, credential collection and administration infrastructure, but recovered artifacts provide uneven evidence for these relationships. This study examines UNK_DeadDrop artifacts collected through 24 July 2026, beginning with three phishing messages. Static inspection, recorded authenticated decryption, repository history, frontend-source analysis and read-only database queries reconstruct the supported relationships. The findings extend the examined public reports with a mutable command-and-control resolver, capture-streaming artifacts, and administration interfaces linking delivery monitoring with wallet operations. Two targeting databases contain 142,365 distinct stored address strings associated with 27,698 repository values. These totals measure collected targeting material; they do not estimate infections or victim losses. The editor-extension and Windows execution branches corroborate mechanisms described in prior reporting. Restricted inputs and absent endpoint and backend telemetry limit independent replication and prevent confirmation of execution, financial loss or operator identity. The case identifies dependencies relevant to detection and incident scoping while distinguishing implemented capabilities from observed outcomes.

Keywords: malware analysis; recruitment phishing; developer security; credential theft; threat intelligence; UNK_DeadDrop

1. Introduction and related work

Recruitment lures expose developers to malicious repositories, editor extensions and terminal commands presented as technical assessments. Published analyses describe these delivery mechanisms, but their relationships to targeting data and administration infrastructure require additional evidence. [1–5]

This investigation began with three phishing messages received on 22 June and 21–22 July 2026. Their links and associated artifacts led to samples using Nodit, Category Labs, D2S AI Trade, Apex Systems, PredictTogether and ECHOES branding. These names identify lure themes; artifact relationships determine whether particular samples belong to a common operation. Shared branding or accounts alone cannot establish identical payloads or common ownership. [E01]

Proofpoint described the repository-task and Visual Studio Code extension-package (VSIX) route, native agents and Windows browser-decryption branch before this study. Vitali's earlier case provides a direct comparison; Jamf, Microsoft and Unit 42 describe related recruitment campaigns. The original contribution comparison covers these five publications, checked on 21 October 2026. The delivery and Windows findings therefore corroborate prior work rather than establish new mechanisms. [1–5]

A bounded public-source refresh through 25 October added an UNK_DeadDrop-specific passive-DNS investigation, a first-person TaikoTalent incident report and its same-origin follow-up material, later Overlord and MALFEX research, and current ATT&CK pages. These sources inform the post-cutoff assessment in Section 11.1; they do not alter the 24 July artifact cutoff or silently expand the frozen case corpus. [6–16]

The additional findings concern configuration resolution, capture implementation, administration interfaces and targeting databases. These details were absent from the five publications examined; no broader claim about unpublished research or product coverage follows. Proofpoint retains UNK_DeadDrop as a distinct cluster, and this study does not establish equivalence with G1052. Behavioural similarities support comparison without resolving attribution. [E03–E08; 1]

1.1 Research questions and scope

This study examines how recovered artifacts connect developer targeting, delivery and collection. It also asks which implementation details extend prior reporting and which conclusions remain unsupported without endpoint or backend telemetry. The analysis combines payload inspection, repository history, frontend source and database aggregates. Its scope is an artifact-based case study, with no estimate of prevalence or malware effectiveness.

2. Materials and methods

The corpus combines delivery and collection artifacts with repository history, administration source and targeting data. Figure 1 summarises the supported relationships and their principal gaps. Appendix B identifies the evidence reports and restricted source records. [E01–E21]

Collection followed the case's static-analysis and passive-retrieval rules, using public resources, research services and public blockchain reads. No sample was installed or detonated, and recovered operator credentials were not used. Historical network collection was not repeated for this revision. Service availability statements therefore refer to the observation date, principally June–July 2026. [E01]

Inspection compared source control flow, configuration, archive contents, binary symbols and file hashes. The abstract syntax tree (AST) analysis parsed JavaScript without evaluating the target. Recorded AES-GCM authentication tied recovered plaintexts to encrypted inputs; that decryption was not rerun for this edition. Hashes identify bytes for comparison but do not establish execution or validate an analyst's interpretation. [E03–E05]

Both targeting databases were opened read-only for counts, an equality join and UNION queries. The latest retained aggregate check, dated 24 October 2026, reproduced all five results without exporting contact rows. Ingestion lowercased and trimmed addresses; the aggregate queries applied no additional normalisation or person-level identity resolution. Counts therefore refer to stored address strings and repository values. Appendix C specifies the queries. [E08]

Artifact observations and runtime outcomes are assessed separately throughout the paper. Source establishes configured actions and control flow; endpoint telemetry is required to confirm execution, and backend or transaction records to confirm subsequent operations. Comments, dependencies and symbols provide narrower evidence than a complete implementation. Each finding identifies the evidence supporting its interpretation and any limitation that changes its scope.

The post-cutoff refresh used the half-open window `(24 July, 25 October 2026]`, bounded public-web search and passive registry or repository access. Search results were treated as leads until the underlying page was retained. Items were classified as direct case updates, corrections, related context, derivative reporting or unverified leads. No listed lure or C2 endpoint was contacted. [6–16]

Command and control is abbreviated C2, and an indicator of compromise is abbreviated IOC. Platform mechanisms include App-Bound Encryption (ABE), the Windows Data Protection API (DPAPI), Component Object Model (COM) elevation, and macOS Transparency, Consent, and Control (TCC). Their effectiveness depends on operating-system, application, policy and session conditions.

2.1 Corpus selection and provenance

The corpus is a case-derived collection assembled from three received phishing messages and the artifacts associated with their links. Follow-up examination covered linked delivery repositories, lure bundles, staged payloads, configuration resources and related administration material. Inclusion reflects documented relationships examined during the investigation; it is not a random sample of recruitment campaigns or a complete inventory of deployed variants. Lure branding alone was not treated as evidence of common ownership. [E01–E03, E07]

The starting messages date to 22 June and 21–22 July 2026, and the historical evidence cutoff is 24 July. Earlier timestamps within repository histories and build metadata describe artifact events rather than their acquisition. October aggregate checks and companion tests are reported separately from historical collection. No single acquisition date is assigned to the entire corpus. [E01, E08, E11–E12, E15, E21]

Variant identification uses SHA-256 and byte size for retained files, supplemented by configuration, source comparisons, archive members and recorded revision identifiers. A matching hash identifies byte-equivalent inputs; different hashes require content comparison before attributing the difference to configuration or behaviour. For example, the two equal-length JavaScript payloads differ in embedded ZIP timestamps rather than configuration. Separate delivery branches and the incomplete ECHOES lineage remain distinct where their payload relationships are unproven. [E03–E05, E11]

Retained inputs include messages, lure JavaScript, the VSIX, encrypted and recovered plaintext payloads, native binaries, repository material, frontend source and two SQLite databases. Availability varies within each evidence report. Some ownership, infrastructure, upstream-comparison and development-network observations are preserved only in analytical notes because the original responses or snapshots are unavailable. The ECHOES final payload and the administration backend were not recovered. These gaps constrain independent checking of the affected claims. [E01–E21]

The supporting reports identify retained inputs by size and SHA-256 and distinguish artifact excerpts from secondary analytical records. Fingerprints describe the bytes reviewed at preparation; they do not authenticate acquisition times or certify an unbroken chain of custody. Selected

excerpts omit credentials, raw contact data and personal attribution leads. The public reports therefore support review of the analysis without releasing a complete forensic corpus.

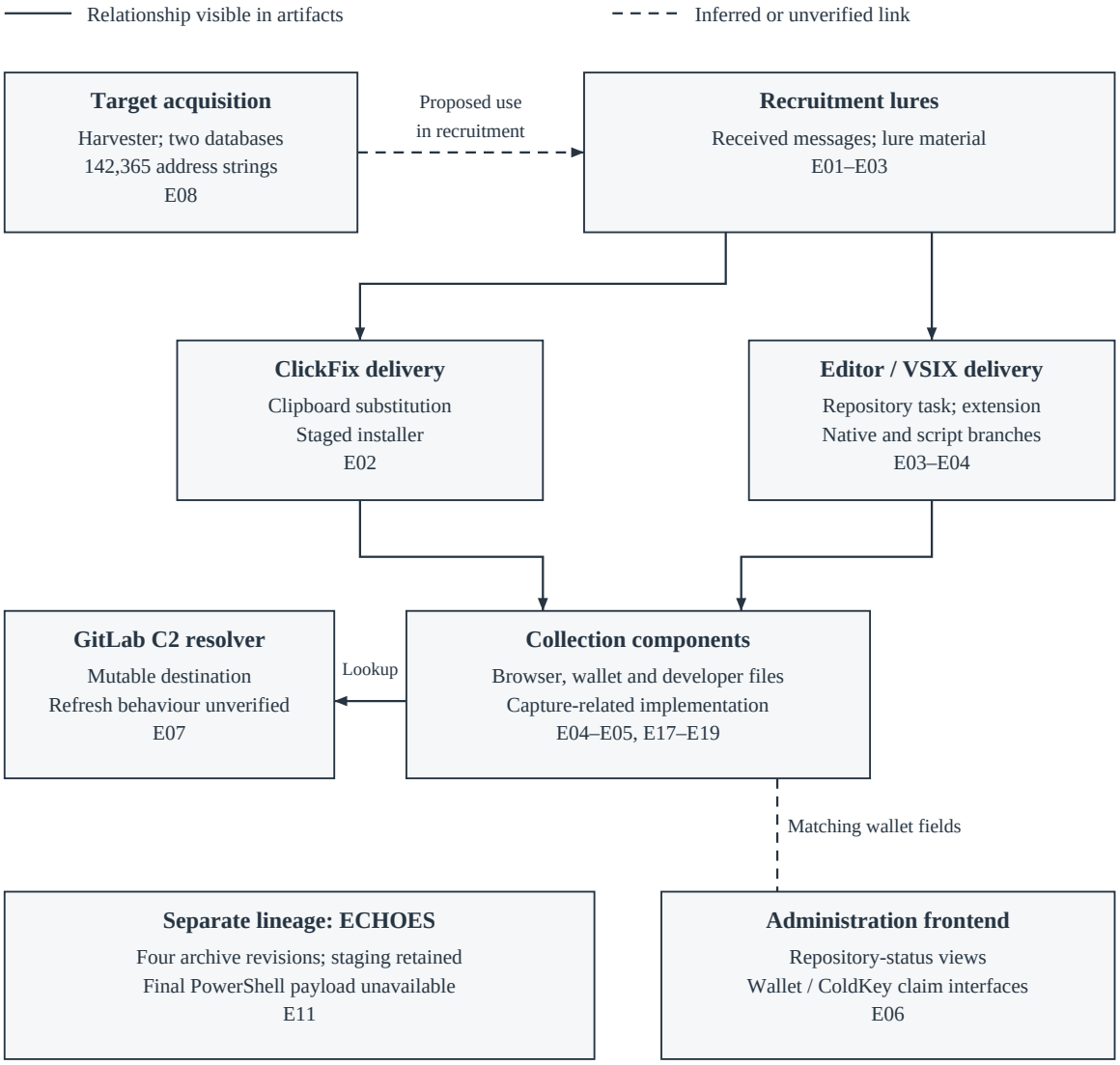


Figure 1. Relationships supported by the recovered corpus. Solid arrows represent relationships visible in source or retained artifacts, not observed execution. Dashed links indicate proposed use of targeting data in recruitment and correspondence between collection targets and frontend wallet fields; delivery and backend transactions remain unverified. The resolver link applies to the inspected payload, not necessarily every component. ECHOES is shown separately because its final payload is unavailable. Evidence IDs refer to Appendix B.

3. Target acquisition and recruitment lures

The recovered harvester queries repositories by star range, visits branches and extracts author and committer addresses from commit metadata. It lowercases and trims addresses before filtering them. Worker configurations partition collection, while scheduler and merge code consolidate results into two databases. A saved scheduler log describes an April 2026 run. [E08]

The analysis note describes sender configurations referencing both databases, with different intervals, cooldowns and warm-up settings. It also records Ondo and NXLog message templates. Those sender configurations are not retained among the exhibit inputs, so their relationship to harvesting remains a reported code observation. Actual delivery volume requires mail records. [E08]

The safe pool contains 79,698 rows and the antispam pool 62,667. Their address fields have no exact overlap under the tested comparison. Their union contains 142,365 distinct address strings

and 27,698 distinct repository values. The repository total corrects an earlier count based only on the safe pool, whose repository set overlaps the antispam set. These counts measure collected targeting material, without resolving unique people, delivered messages or compromised endpoints. The three received messages provide separate evidence of delivery. [E01, E08]

Mail-provider relationships derive from configured hosts and message headers rather than variable names. The guide describes Namecheap Private Email settings, while the analysis record reports Mailgun-related evidence in later messages. The retained guide also discusses spam filtering and a shared outbound relay. That relay supports a provider association without linking unrelated tenants to the campaign. [E09]

Lure material includes company impersonation, portfolios, testimonials, aged accounts and functional-looking projects. The clean PredictTogether source differs from its weaponised deployment, making the delivered repository, archive and revision the relevant unit of inspection. Four payload-bearing forks and public recruitment replies provide exposure leads, but do not establish task execution or extension installation. One repository initially classified as victim-owned was subsequently suspected of operator staging after comparison of creation times and related artifacts. Provider records would be needed to resolve that ownership hypothesis. [E01–E03, E10]

4. Delivery mechanisms and payload staging

The ClickFix branch embeds a verification step in a recruitment-themed web application. Its copy handler substitutes installation logic for the displayed command while preserving a plausible verification result. This design allows the verification interaction to accompany a background installation attempt. Device checks and command builders vary across the recovered bundles. Unix-like commands use detachment and output suppression; Windows commands use hidden-window execution. Some copied strings include a trailing newline, whose effect depends on terminal paste behaviour. The handler establishes clipboard substitution, but terminal execution was not reproduced. [E02]

The retained ClickFix chain has three stages. The first-stage samples contain approximately 41 KB of shell or 39 KB of batch content, including an encoded installer and installation-status reporting. Verification, bot-check and design-check filenames recur across related droppers. Stage two handles prior instances, removes stale files, configures primary and fallback stores, and can stage Node.js 20.18.1. An official-distribution fallback reduces dependence on an existing Node installation. The macOS branch removes the downloaded runtime's quarantine attribute. Retrieval still depends on the configured locations being reachable. [E01, E02]

Stage three contains an AES-256-GCM envelope comprising a 12-byte nonce, a 16-byte authentication tag and ciphertext. The loader uses an embedded key, decrypts in memory, invokes Node's module-compilation machinery and schedules deletion of its own file. Two recovered plaintexts are each 1,140,214 bytes but have different hashes. Content comparison attributes their differences to timestamps for two package-lock members inside embedded Windows bundles; the configuration is identical. [E01, E05]

The repository route uses editor metadata to invoke an installer. A `runOn: folderOpen` task points into a plausible project directory, while labels and cleanup reduce conspicuous material remaining in the workspace. The retained VSIX is 17,647,582 bytes and identifies itself as `google-dev-tools.google-update-support`. It contains native agents matching published hashes and declares startup activation. Workspace settings supply the server URL, client identifier and token, allowing package reuse across deployments. Execution depends on editor trust, task permissions and platform-specific control flow. Extension persistence after project deletion does not establish identical startup behaviour across operating systems. [E03; 1]

The ECHOES archives form a separate, incomplete delivery lineage. Four revisions preserve lure and encoding structure while changing repository-based retrieval configuration. The third

revision adds VBS persistence and an elevation request; the fourth changes only the destination and token. The referenced final PowerShell stage was unavailable through the examined unauthenticated routes and was not recovered. Archive continuity therefore supports staging relationships without establishing the missing payload's capabilities. [E11]

5. Static deobfuscation and platform-specific execution

The VSIX loader used string-array indirection that prevented simple literal recovery. The recorded Acorn workflow parsed wrapper functions and mapped their decoder indices and offsets without evaluating the target. Candidate rotations were compared across resolved call sites. The selected rotation produced printable decodes at all fifteen tested sites, compared with three for the next candidate. Key reconstruction preserved the source order of nine decoder results and seven literal fragments. [E04]

The resulting key authenticated three payloads: the credential-handling Python component, the Windows Node agent and its orchestration script. The plaintexts contain readable source and comments, but successful authentication provides the basis for the recovery claim. This procedure corroborates the key and Windows architecture already described by Proofpoint. [E04; 1]

The Windows collection branch uses Node.js and Python without requiring the native Go agent. It can run the editor's Electron executable under `ELECTRON_RUN_AS_NODE=1`, allowing a legitimately signed binary to interpret an unrelated collection script. Native-agent hashes alone therefore do not cover this branch. Process arguments, ancestry and script provenance provide additional detection context. [E04]

5.1 Privilege handling

The recovered Windows source constructs PowerShell that requests an elevated COM object and invokes its shell-execution method through a runtime-created delegate. Interface identifiers are assembled from numeric fragments. Similar CMSTPLUA-related builder logic appears in the VSIX and ClickFix branches, supporting shared tooling. The intended child process is the editor executable acting as a Node interpreter; fallback behaviour is also present. [E19]

This source establishes an elevation attempt without demonstrating a prompt-free outcome. Successful activation depends on policy, integrity level, user group and component registration. Correlated COM activation, process ancestry and browser-data access would distinguish an attempted launch from completed elevated collection. [E19]

Linux and macOS branches can solicit passwords through desktop dialogs and test submitted values using `sudo`. Recorded loops allow repeated prompts for approximately ten minutes on Linux and thirty minutes on macOS. Accepted input can support subsequent credential recovery and persistence. These paths were not reproduced on representative endpoints for this edition. [E01, E04]

5.2 Framework components and sample-specific modifications

The Go implant derives from the public Overlord client framework. Its module path, WebAssembly runtime, remote-access functions and transport configuration identify software lineage without identifying an operator. The reported upstream comparison did not locate the sample's build revision in the public history examined. This alone cannot distinguish an unpublished clone from pruned history or another metadata explanation. [E18]

The comparison identified SQLite-backed browser collection and wallet-related modules absent from the selected upstream files. The Windows JavaScript/Python branch, staged collection and GUS configuration namespace provide additional evidence of deployment-specific logic. The original upstream snapshot is not retained, and the comparison does not cover every historical version, fork or plugin. [E18]

Capture evidence also differs by artifact type. Selected capture and audio dependencies were absent from one Linux module listing, while later binary inspection identified capture and frame-difference symbols. Dependency declarations, linked routines and reachable runtime paths answer different questions. The combined record supports capture-related implementation with build-dependent packaging; representative runtime tests would be needed to establish which builds provide working capture. [E05, E18]

6. Credential collection and screen capture

Browser collection targets Chromium profile files, Firefox `key4.db` and `logins.json`, and platform-specific credential stores. The Windows Python component includes DPAPI handling and logic intended to obtain ABE material through browser elevation interfaces. Staging paths and comments describe attempts to satisfy caller-path requirements. Unix-like branches target Safe Storage, desktop keyrings and macOS Keychain material. [E01, E04]

Collection phases stage browser and wallet artifacts before attempting additional credential recovery. An elevated process does not automatically obtain secrets protected by the original user's session. Recoverability depends on browser version, policy, session state and execution context. [E04]

Wallet selectors include 35 extension identifiers, standalone wallet paths, mnemonic-related filenames and Bittensor ColdKeys. ColdKey and mnemonic fields also appear in the administration frontend, linking collection targets to intended wallet workflows. Developer-secret selectors cover SSH material, cloud and package-manager configuration, environment files and private-key extensions. These selectors extend potential exposure to source control, package publication and cloud administration, subject to file presence and access. [E01, E06]

The `.kdbx` extension appears in a generic file-selection set. No dedicated KDBX parser or demonstrated master-password attack was identified in the reviewed material. Collection of such a file would expose an encrypted database without establishing recovery of its protected contents. Negative searches for other password-manager names do not establish that those products are unaffected. [E05]

Screen-capture evidence combines binary symbols, dependencies and installer checks. The Go implant contains JPEG encoding, previous-frame state, block comparison, and frame-rate and bandwidth telemetry. Symbols including `CaptureAndSend`, `encodeBlocks`, `blockChanged` and `copyPrev` support a design consistent with transmitting changed image regions. The Windows bundle contains a native capture dependency, JPEG encoding, MessagePack and WebSocket support. Installer checks can reject bundles missing the capture module; the associated task is `GoogleUpdateSupporterNode`. These artifacts support a capture-streaming implementation, without providing a measured frame rate or recorded session. [E03, E05]

The Windows bundle also contains synthetic input-injection and clipboard modules. Input injection is distinct from keystroke capture, which was not established. Apparent keylogging matches included Go TLS key-log facilities. Negative searches for keyboard hooks, shorteners and other family names remain bounded by the examined material and cannot exclude dynamically supplied components. [E05]

6.1 Collection phases and filesystem scope

The stage-three JavaScript accepts main, pass and phase3 upload variants. These labels do not define exclusive classes of collected data. The third branch traverses the home directory using filename and extension selectors. Terms include keys, wallets, phrases, passwords, seeds, mnemonics, secrets, credentials, backups, recovery data, keystores and tokens. Candidate formats include JSON, YAML, TOML, INI, property lists and environment files. Selection is lexical; it does not identify every secret within arbitrary file contents. [E17]

Recorded defaults limit traversal to eight minutes overall, two minutes per root and fifteen seconds per directory read. Additional caps allow 2,500 directories, depth six and 500 candidate files. Environment variables can override several limits, while system, cache, dependency and cloud-storage locations are excluded. These bounds describe configured scope rather than the extent of collection on an observed host. [E17]

On macOS, readability probes precede access to selected protected directories. Failed or timed-out probes cause those directories to be skipped. This behaviour supports conditional prompt avoidance, without establishing a universal TCC bypass. Previously granted access and changing permission states can alter the outcome. [E17]

6.2 Data transfer and configuration

Uploads use chunks of two mebibytes by default, with bounded retries per chunk. A separate retry loop begins at five-second intervals and increases to a two-minute cap. It is designed to continue until the work completes or execution ends. Successful transfer still depends on connectivity, accepted credentials and server responses. The inspected request path disables TLS certificate verification, removing server-authentication checks without establishing interception or successful upload. [E17]

Build-time placeholders identify a server address, agent token and client seed, with fallback values if substitution does not occur. GUS settings also control traversal budgets, retry behaviour and chunk sizing. Substitution and environment overrides can change effective settings between deployments. Sensitive bearer values are omitted from the public evidence reports. [E17]

7. C2 configuration and infrastructure history

A retained payload resolves its C2 address through `gitlab[.]com/drop-indexing/tasks/-/raw/main/config.json`. This dependency separates destination selection from the distributed payload. The recorded history includes configuration rotation associated with a July domain batch. A client consulting the updated resource could select a different server without any change to its file hash. [E07]

7.1 Resolver dependency and containment

The configuration object, hosting account and version history supplement payload hashes and resolved addresses as investigation targets. Runtime refresh frequency, failure caching and use of the resolver across components remain unverified. Blocking a destination may leave replacement resolution available, while blocking the resolver may leave a cached address usable. These conditional failure modes support monitoring resolver access and subsequent connections together. They do not establish that a particular containment action failed. [E07]

7.2 Historical service associations

The July topology assigns C2 and administration roles to `207.241.172[.]222`, lure hosting to `170.205.31[.]182`, and an earlier C2 role to `23.137.105[.]75`. These assignments combine certificate, passive-DNS and content observations. Registration, retrieval and service disappearance describe different events and do not establish a continuous or exhaustive infrastructure inventory. Shared hosting and CDN relationships do not justify attributing neighbouring assets to the operation. [E01, E07]

The origin analysis reports a certificate and management-panel response associated with `207.241.172[.]222` through third-party scan records. Those raw responses and the certificate were not retained. The available panel HTML and JavaScript were retrieved from `api.domatisc[.]ink`, rather than the bare IP. They establish retained panel content but cannot independently reproduce the reported direct-IP response. The association therefore remains historical and does not establish current availability, exclusive control or operator identity. [E20]

Scan coverage also constrains negative findings. A short port list does not exclude other services, and an empty vulnerability lookup does not establish host security. Common OpenSSH banners and handshake fingerprints provide weak attribution signals. Host-key reuse is more specific but remains compatible with cloned images or compromise. Certificate validity dates the certificate rather than the campaign or host tenancy. [E20]

Mail-tenant values and repeated account identifiers provide further leads for provider-side comparison. Shared defaults provide weaker evidence and should not be treated as unique identifiers. Historical certificate and resolver monitoring adds dated observations without establishing that monitoring remains active. Similarly, empty results for newer domains in selected feeds describe coverage of those queries, rather than an absence of protection across vendors. [E01]

Successive topology snapshots may reflect rotation, parallel roles or changing observation coverage. Their relationships should retain timestamps and types, such as observed resolution, presented certificate, matching content or configured destination. Combining these observations into one undated graph would imply simultaneous activity unsupported by the record. [E21]

8. Administration interfaces

Recovered frontend source from a development deployment of `rms-platform` combines message metrics, database views, task boards, repository monitoring and wallet operations. Repository views expose clone URLs, availability status and refresh controls. Wallet interfaces accept imported material, display or derive addresses, request balance previews, and expose single or bulk claim actions. Chain types, mnemonics and ColdKeys correspond to endpoint collection targets. Together, these features connect delivery maintenance and intended wallet handling within the recovered application. [E06]

The backend was not recovered, so routes and field names do not establish operational endpoints, populated databases or enforced privileges. Client-side role checks cannot establish exclusive control of privileged actions, and claim controls do not establish completed transfers. Repository-monitoring controls likewise show intended functionality without proving that suspensions produced alerts or rotations succeeded. [E06]

The analysis places recipient selection on the server side. Endpoint execution alone would therefore not necessarily reveal the destination of a later wallet operation. Backend evidence, additional configuration or confirmed transactions would be needed to resolve it. The separately recovered guide describes server setup, mail configuration, GitHub-user collection and deliverability procedures. Text resembling copied assistant output is consistent with procedural reuse, but neither that text nor signup interfaces measures team size or operator skill. [E06, E09]

9. Attribution leads and financial context

Repository history, commit metadata, mailbox patterns, domain records and collaboration artifacts provide attribution leads of unequal strength. Reuse of specific artifacts or credentials among linked samples carries more information than similar public names. A workstation path embedded in a Python docstring can support provenance enquiries, but is copyable and cannot authenticate a developer's identity. Configured time offsets and apparent business-hour patterns likewise do not establish physical location or nationality. A cgo-enabled `darwin/arm64` build does not uniquely identify its build machine. [E01, E12]

Financial observations do not establish victim cash-out or control of an exchange account. The separate development-network application is not the administration wallet console and does not establish mainnet theft. Its deployed bundle and raw chain-query responses are unavailable among the exhibit inputs, leaving those observations dependent on analytical notes. The encrypted environment archive does not independently establish the deployed configuration or its sealed contents. [E13]

The ECHOES poster provides a further provenance lead through embedded image-generation assertions. These describe an AI-image pipeline but do not identify the person who created or deployed the artwork. No independent signature-verifier output is retained; embedded ingredient-validation assertions are not a substitute for such verification. Certificate validity alone also cannot date image creation. [E14]

9.1 Build chronology and timestamp interpretation

The chronology combines mail headers, domain registrations, configuration commits, ZIP-entry timestamps and Go version-control metadata. A message Date header records 01:08:32 UTC on 21 July, followed five minutes and thirty-four seconds later by the implant's recorded `vcs.time`. The latter identifies a source-revision timestamp rather than certified compilation time. Their proximity supports a temporal association without proving that compilation accompanied message delivery. [E21]

The retained Windows ZIP includes members dated 13 July and two package-lock members dated 20 July. Its complete member-date range spans 18 June to 20 July. Stored local timestamps may reflect build settings, repackaging or deliberately assigned values. Comparison with UTC fields can identify a configured offset, without locating an operator. The Go `trimpath` setting also limits path information without identifying a build host. [E12, E21]

Chronological interpretation therefore retains each field's source, timezone, mutability and event type. A commit, file modification, registration and mail receipt remain separate events even when their times are close. Provider records or independent host artifacts would be needed to connect those events to particular accounts or machines.

10. Detection and incident response

Detection benefits from combining branch-specific process context with collection and configuration markers. Framework names, folder-open tasks and updater labels occur in benign or unrelated software. Wallet-upload strings, unexpected extension installation and resolver access provide more specific context when correlated with process ancestry. ClickFix begins with a terminal interaction, whereas the VSIX route depends on editor tasks and extension behaviour. A single editor-to-shell sequence therefore does not describe both branches. [E02–E05, E15]

The companion YARA rule combines framework and environment markers with multiple wallet-related strings. Two Sigma rules select the recorded Linux systemd unit path and Windows creation of the capture task through `schtasks.exe`. Renamed components, changed strings and alternative task-registration APIs remain coverage gaps. These experimental selectors have not undergone a fleet-wide precision or recall evaluation. [E15]

The companion AST scanner reports selected clipboard calls and campaign markers with source locations and input hashes. It folds limited literal expressions while leaving dynamic values unresolved. Eight AST tests passed in an offline rerun on 24 October; four detection-test passes remain historical results from 21 October. Earlier spot checks covered two JavaScript files and a Go implant. These checks establish tested behaviour, without validating general deobfuscation, production accuracy or resistance to parser resource exhaustion. [E15]

10.1 Endpoint event correlation

Relevant sequences include repository opening, task execution, script or runtime retrieval, and an editor executable subsequently acting as a Node interpreter. Browser-database access, wallet-file access and outbound transfers can provide evidence of later collection stages. Individual events remain ambiguous because developer tools routinely launch processes and access project data. Correlation should account for delivery branch, operating system and alternative event ordering. [E02–E05, E19]

Detection records should distinguish artifacts, attempted actions, completed actions and observed outcomes. A task definition does not establish execution, and a wallet selector does not establish that a mnemonic was present. A capture-task name similarly requires runtime evidence before it supports an exposure assessment. These distinctions determine which additional telemetry is needed for incident scoping.

10.2 Incident scoping and response assessment

Scoping should cover installed extensions, services, scheduled tasks and staged runtimes beyond the lure directory. Credential exposure depends on the executed component, collection phase and accessible files. Browser sessions, password-manager databases, cloud tokens, SSH keys and wallet material consequently require different assessments. Screen exposure additionally depends on which capture component ran and when. [E01, E04, E05]

Network investigations should retain resolved addresses alongside resolver dependencies and observation times. Historical DNS and TLS records support retrospective reconstruction; current blocking decisions require current telemetry and ownership checks. Shared provider relationships alone do not justify blocking an entire range. [E07, E20]

The response notes describe one preservation request as filed and others as drafted or proposed. No provider acknowledgement was identified in the reviewed exhibits, and this revision performed no outreach or status refresh. Request delivery, provider review, suspension and reduced victimisation are separate outcomes. The available records do not establish measured disruption. [E01]

11. Discussion and limitations

The combined artifacts connect targeting, delivery maintenance and collection within a broader administration workflow. The resolver shows how destination selection can change independently of payload bytes, while wallet interfaces correspond to selectors in collection code. These relationships extend the examined literature beyond the already reported VSIX and Windows mechanisms. Their contribution is an implementation-level account of dependencies, with no inference of successful end-to-end operation. [E06–E08; 1–5]

The evidence varies substantially in completeness. Retained source supports control-flow inspection, whereas symbols and dependencies support narrower capability assessments. Some infrastructure, upstream-comparison and development-network findings survive only in analytical notes. The ECHOES final payload remains unavailable, and authenticated decryption is reported from the original record. Restricted inputs further prevent independent public replication of the database totals and several artifact analyses.

Alternative explanations constrain attribution and impact claims. Shared hosting did not sustain an alleged earlier campaign phase, and repository timing changed an ownership hypothesis without resolving it. Empty RPC results did not establish wallet inactivity, while TLS key-log symbols did not establish keystroke capture. These reassessments preserve the artifact observations while narrowing the conclusions that follow from them. [E05, E10, E13, E16]

The main unresolved questions require different forms of evidence. Representative endpoint telemetry would test resolver refresh, capture availability and platform-specific execution. Backend records or confirmed transactions would distinguish intended wallet operations from completed transfers. Provider-side records would help assess account control and infrastructure relationships. None of these outcomes can be inferred from the targeting totals or frontend interfaces alone.

Technique mapping requires the same care. A wallet-claim interface does not establish financial theft, and encrypted payload storage does not establish encrypted C2 transport. Similar recruitment themes also do not justify assigning the observed behaviours to another group's profile. The paper makes no claim to revise the current ATT&CK corpus. [E01]

11.1 Post-cutoff public-source update

Hopkins's UNK_DeadDrop-specific passive-DNS investigation was published on 23 July and updated on 16 August. It reports `api.chestsoi[.]ink` in the same GitLab dead-drop sequence examined here. Because the original publication predates the evidence cutoff, this is a coverage correction rather than a post-cutoff publication. The retained July case capture does not preserve that resolver generation, so the domain remains a direct-source lead rather than a silently merged historical IOC. [E07; 6]

On 9 October, a first-person TaikoTalent report described a recruitment lure whose launcher used the `deepfake_guard` name, the same POSIX double-fork form, `/var/tmp/.kdm.XXXXXX`, and `/guard-launcher-result`, with paired primary and fallback `.ink` hosts. That combination is sufficiently distinctive to treat the incident as a probable UNK_DeadDrop continuation lead. The article, Gist, repository and mailing-list follow-up nevertheless share one reporting origin; the reported sample was not independently obtained in this refresh. A list moderator separately criticised the disclosure's AI-assisted presentation and unsupported claim of prior list disclosure while allowing that the incident may have been real; the author subsequently acknowledged the disclosure and process problems. The new domains and hash therefore remain research leads rather than independently verified operational indicators. [E01, E02; 7–11]

Later WatchGuard reporting documents an Overlord deployment through Brazilian legal-notice lures and a Windows VBS chain, while SafeDep describes the MALFEX `imgdecode` native Node add-on. A September MalwareBazaar record supplies another Overlord sample with distinct infrastructure. These sources extend framework and ecosystem context but do not establish shared operator or campaign identity with UNK_DeadDrop. [E18; 12–14]

The current G1052 page, last modified 31 July, did not list `T1102.001`, `T1566.002` or `T1552.001` when checked on 25 October; the separate T1102.001 page likewise did not name G1052. This is a bounded page-state observation, not evidence that MITRE considered and rejected the case mappings. [15, 16]

12. Conclusion

This case study extends the examined UNK_DeadDrop reporting with resolver history, capture-streaming artifacts, administration workflows and a reproducibly counted targeting corpus. It also corroborates known delivery branches and documents the recorded recovery of their encrypted contents. A post-cutoff source refresh adds a probable continuation lead without changing the frozen historical corpus or resolving attribution. These findings identify implementation dependencies relevant to detection and incident scoping. Confirmation of execution, backend operation, financial loss and operator identity remains dependent on evidence beyond the recovered corpus. [6–16]

Data availability and ethical handling

The underlying archive is restricted because it contains malware, harvested contact data, credentials, provider identifiers and unverified identity leads. It is not deposited with this preprint. Supporting reports E01–E21 present edited observations, selected exhibits, methods, limitations and source fingerprints; Appendix B lists their roles. They are distinct from the public references 1–16 and do not constitute independent replication. No public repository URL or archival DOI has been assigned to the companion tooling.

Public materials include aggregate counts, sample hashes, defanged historical indicators and analytical descriptions. Raw contact rows, usable credentials and personal attribution leads are excluded. Public-interface collection does not itself establish consent from people represented in the recovered data. No sample execution or recovered-credential use occurred during this editorial revision.

Appendix C makes the aggregate method inspectable, but public replication requires access to the withheld databases. Hashes permit comparison by readers who independently possess the relevant files. The companion's checks are engineering validations rather than a performance benchmark. Historical acquisition and decryption remain attributed to the original case records.

References

[1] Proofpoint. Don't Fear the Repo: UNK_DeadDrop Phishing Campaign Targets Developers to Steal Cryptocurrency. 8 June 2026.

[2] Denys Vitali. I was likely targeted by DPRK in a sophisticated developer malware campaign. 25 May 2026, with June update.

[3] Jamf Threat Labs. Threat Actors Expand Abuse of Microsoft Visual Studio Code. 20 January 2026.

[4] Microsoft Defender Experts and Microsoft Defender Security Research Team. Contagious Interview: Malware delivered through fake developer job interviews. 11 March 2026.

[5] Unit 42. Contagious Interview: DPRK Threat Actors Lure Tech Industry Job Seekers to Install New Variants of BeaverTail and InvisibleFerret Malware. 9 October 2024.

[6] Ben Hopkins. Hunting UNK_DeadDrop - Uncovering the Foxtopia Stealer Through Passive DNS Pivoting. 23 July 2026; updated 16 August 2026.

[7] Edison Flores. A fake recruiter sent me a job offer that ended in my terminal — the trojan is still live. 9 October 2026.

[8] Edison Flores. taikotalent.xyz fake-recruiting terminal trojan campaign — IoCs, hashes, attack chain. Public Gist; checked 25 October 2026.

[9] Edison Flores. FOLLOW-UP: taikotalent.xyz fake-recruiter trojan — RESOLVED. 15 October 2026.

[10] Solar Designer. Re: FOLLOW-UP: taikotalent.xyz fake-recruiter trojan — RESOLVED. 15 October 2026.

[11] Edison Flores. Re: FOLLOW-UP: taikotalent.xyz fake-recruiter trojan — RESOLVED. 17 October 2026.

[12] Euler Neto, WatchGuard Threat Lab. How VBScript Malware Hides Behind Trusted Tools to Evade Detection. 22 October 2026.

[13] SafeDep Team. MALFEX Moves Its npm Loader into a Native Windows DLL. 13 October 2026.

[14] MalwareBazaar. SHA-256 2da7c151676e991ebee73da26db28e6872f655b5487dc4b424a6024a3381472a (OverlordRAT). First seen 8 September 2026; checked 25 October 2026.

[15] MITRE ATT&CK. Contagious Interview, Group G1052. Version 1.0; last modified 31 July 2026; checked 25 October 2026.

[16] MITRE ATT&CK. Web Service: Dead Drop Resolver, T1102.001. Version 1.2; last modified 11 September 2026; checked 25 October 2026.

The five-source contribution comparison was checked on 21 October 2026. A separate bounded refresh added 6–16 on 25 October. It did not re-open the historical evidence cutoff or contact listed lure or C2 infrastructure. The new web references were retained in a dated Noesis refresh; dedicated citation- archive operations were unavailable to the current principal, so the paper does not claim an independent archival deposit.

Appendix A. Sample hashes and historical indicators

The held VSIX has SHA-256 `52886aab179f26421678ff23af1b0fabf0a17ffbb534369cdbbac8008cbed8e7`. The Go implant used in the companion's static YARA spot check has SHA-256 `ef183bbb40f4c7c3f4b52994cdba3295f3eab2604fdad99eefa3b67b8d966e8d`. The 1,140,214-byte JavaScript payload has SHA-256 `694626bed2df1f0d0026a3f7ead7692b44c7e73041cdd799b79a025335533cdc`. The lure JavaScript used in the clipboard-call spot check has SHA-256 `5ee915bb8d0363bd92176cc595d7425fd7e7830508d8824d90b5285a3d2b11bd`. These identify specific held files and should not be read as complete family coverage. [E03, E15]

Historical July lure or campaign observations include `apexsyshire[.]ink`, `d2saitrade[.]ink`, `joincategory[.]xyz`, `3dscenebox[.]online`, `surgecareer[.]ink` and `validator.aavzx[.]ink`. The record also associates `cklomino[.]ink`, `dsfmot[.]xyz`, `aazdias[.]ink` and `fallaazdis[.]ink` with backend or store roles. These are dated research indicators, not a current blocklist. Infrastructure associations have different evidentiary roles; association alone does not establish that an asset is malicious. [E01]

Post-cutoff research leads are kept separate from those historical indicators. They include `api.chestsoi[.]ink`, `taikotalent[.]xyz`, `api.gaganata[.]ink`, `api.fallgganata[.]ink` and reported SHA-256 `c7c22e322c17bda8198257342d4034af6250a5c433242461b870879ba9aa285d`. The TaikoTalent values come from one reporting origin and the sample was not independently recovered in this refresh. These entries are research leads, not a current blocklist or an extension of the 24 July evidence corpus. [6–11]

Useful content and host artifacts include the extension identity, `GoogleUpdateSupporterNode`, `google-agent.service`, wallet-upload endpoint strings, the resolver path and the recorded collection selectors. Operator credentials, harvested addresses, personal contact details and provider account identifiers are intentionally omitted to protect sensitive data.

Appendix B. Supporting evidence register

The supporting reports E01–E21 were prepared on 24 October 2026 from restricted case records, principally from June–July 2026, and the October companion engineering record. Each report presents the relevant observations, methods and limitations and identifies its source records by descriptive title and SHA-256. They are edited analytical reports, not independent literature sources or original forensic exports. The underlying restricted archive is not included.

[E01] Master investigation report. Scope, staging, infrastructure, corrections and historical response record.

[E02] Clipboard-lure deobfuscation analysis. Recovered handler, command construction and telemetry.

[E03] Editor-extension package analysis. VSIX identity, configuration, native-agent hashes and build differences.

[E04] Authenticated payload-decryption analysis. Recorded AST-assisted key recovery, authentication and Windows source.

[E05] Payload capability re-examination. Capture artifacts, selectors and bounded negative searches.

[E06] Administration frontend analysis. Recovered repository-monitoring and wallet-management interfaces; no backend.

[E07] Configuration and infrastructure history. Resolver dependency and historical hosting associations.

[E08] Target-acquisition code and two SQLite databases. Harvester implementation and read-only aggregates; queries in Appendix C.

[E09] Recovered deployment and mail guide. Mail-provider configuration and workflow interpretation.

[E10] Repository-ownership reassessment. Corrected ownership hypothesis and comparison with published indicators.

[E11] Archive lineage and configuration rotation. Four archive revisions and an unrecovered final stage.

[E12] Attribution and build-artifact notes. Identity and chronology leads; no independent personal attribution.

[E13] Separate development-application analysis. Development-network context; not proof of wallet-console recipients or theft.

[E14] Image-provenance examination. Recorded content-provenance analysis; no fresh signature verification.

[E15] Companion tooling and validation record. Parse-only AST scanner, experimental rules, an offline AST-test rerun and historical detection tests; no public release identifier assigned.

[E16] Earlier-phase retraction. Withdrawal of an attribution based on shared hosting.

[E17] Stage-three collection and configuration analysis. Filename selectors, traversal limits, directory checks and transfer mechanics.

[E18] Framework comparison. Comparison with selected public upstream files; not an exhaustive historical audit.

[E19] Windows elevation and Electron analysis. Static COM elevation attempt and script execution path; no runtime validation.

[E20] Historical origin observations. Analytical notes describing third-party scans and certificates, plus retained domain-served panel files; raw scan responses unavailable.

[E21] Build and hosting chronology. Timestamp semantics and evolving topology; fields do not independently certify compile times.

Appendix C. Aggregate verification method

Open both databases in SQLite read-only mode and attach the antispam pool as `other`; the safe pool is `main`. The relation is `emails`, with fields `email` and `repo`. Do not export row-level data. Ingestion lowercased and trimmed addresses; these queries apply no additional normalisation or person-level identity resolution. All five aggregates were reproduced read-only on 24 October 2026, as recorded in E08. The queries and results are:

`SELECT count(*) FROM main.emails;` — 79,698.

`SELECT count(*) FROM other.emails;` — 62,667.

`SELECT count(*) FROM main.emails s JOIN other.emails a ON s.email = a.email;` — 0.

`SELECT count(*) FROM (SELECT email FROM main.emails UNION SELECT email FROM other.emails);` — 142,365.

`SELECT count(*) FROM (SELECT repo FROM main.emails UNION SELECT repo FROM other.emails);` — 27,698.

An address string is the unit of analysis. Repository values are stored references, not a verified count of extant repositories. SQL equality and UNION semantics apply to the stored values; the counts do not resolve aliases, ownership or successful delivery. Re-execution requires authorised access to the restricted inputs.